

Miles Urquidi, 85563092 33%

Daniel Vo, 82561950 33%

Jason Nguyen, 19181867 34%

Miles: Prompt 1 (black box testing)

Dan: Prompt 2, paste the full TriangleType.java code, then: "Here's the implementation. Write at least 5 JUnit tests that maximize branch coverage. List which branch each test hits." then screenshot as prompt2.png

Jason : Prompt 3, "Act as a QA engineer trying to break this triangleType(int,int,int) method (1=scalene, 2=isosceles, 3=equilateral, 4=not a triangle, 5=out of bounds). Write at least 5 JUnit tests targeting only edge/invalid cases: zero, negatives, the boundary where two sides sum to exactly the third, values over 1000, and integer overflow." Save your screenshot & the test code ChatGPT gives.

Paragraphs:

- 1) One summary paragraph on what changes you observed in the generated tests as you tried different prompts.
 - a. The three prompts changed how much of the program ChatGPT has access to, and the tests changed along with it. Prompt 1 just gave a rough scope with zero code, so ChatGPT had to assume and produce mostly normal case tests and a few guesses with the edge cases. It had an incorrect assumption that a negative or zero side would return 5. Prompt 2 included the full source code and asked for branch coverage. Leading the tests to be labeled by branch and the invalid input cases were correct since it could read code. Lastly prompt 3 gave the signature and return codes but no source and told it to "break" the method so every test was an edge or invalid case. Similarly to prompt 1 it wrongly assumed zero/negative returns 5. The amount of details and implementation we revealed was the main difference between each one. More detail meant more correct expected values and more branch targeted tests, while less detail meant broader but partly incorrect guesses.

2. One paragraph summarizing the changes and issues you had to resolve to execute the ChatGPT-generated test cases.
 - a. All the ChatGPT-generated test cases produced were written with JUnit 5 while TriangleType.java and our other test cases from the other homeworks were written to be compatible with JUnit 4. Because of this incompatibility between test cases and previous files, our generated test cases were unable to compile and run. This issue across all our test cases were easily fixed however, with all that was needed to be done were to update the imports at the top to be JUnit 4 compatible imports. Then once updated, our tests ran without problems.
3. One paragraph summarizing code coverage difference between your written test cases (as part of HW2 and HW3) and those generated by ChatGPT.
 - a. We compared branch coverage of all 15 ChatGPT tests with our combined HW2 and HW3 Tests. Counting the program's decision outcomes, the human tests reached every branch and condition while the ChatGPT set reached much less, missing about 4. ChatGPT missed the two valid isosceles branches where the equal pair is side 1 and 3, or sides 2 and 3, because it only ever generated the side1==side2 form of an isosceles triangle. It also only placed a zero or negative value in the first argument, so it never tested the second and third position checks in the side <=0 guard, in composition our tests covered all three positions. ChatGPT hit every distinct return value but not all the paths to it, the human side reached them all by systemically permuting which side was equal or invalid.
4. One paragraph summarizing the presence or absence of any two test smells of your choice.
 - a. Two test smells that are largely absent, in which the test suite avoids well are the Mystery Guest and Assertion Roulette test smells. Assertion roulette occurs when a test makes multiple assertions without giving a clear indication on what failed, which could make debugging difficult. Similarly, the Mystery Guest smell occurs when a test depends on certain files, databases, or has a hidden setup which can make testing

difficult as well. Thus, we were able to avoid it since our test inputs and expected outputs are all explicitly defined within each test case. Also, In our test suite, the tests only focus on a single edge case each and contain only one assertion, helping us avoid Assertion Roulette. Overall, we have an easier time diagnosing failures, and have a clear time figuring out the purpose of each test and its maintainability.

5. One paragraph summarizing the strengths and weaknesses of both sets of test cases.
 - a. ChatGPT's strengths were mainly its speed and how readable the code was. It made test cases that correctly compile and were well named quickly, and when given the source code in prompt 2 it mapped very nicely to branches. Its weaknesses were pretty apparent when it couldn't see the code, as it confidently asserted wrong expected values for zero and negative inputs, so 2 of the 15 failed, and it clumped wrong values in the first argument and missed some branches. Prompt 3 had a few misleading tests as the integer overflow tests having the >1000 guard returns 5 before any addition would happen, so it passes for the wrong reason and never actually tests overflow. Our human tests were slower to write but much more thorough. They reach full branch coverage and the five HW2 tests that fail actually expose a real difference. While their main weakness is how it's meant to be maintained, as there's multiple duplicated functional tests in both HW2 and HW3. Overall, ChatGPT is cleaner and faster but is weaker on edge coverage and sometimes incorrect. The human test cases are more redundant but cover more paths and catch real bugs.